

Vrije Universiteit Amsterdam
Computational Thinking
Project Assignment: iTrade

Group 8

Members with student numbers:

Arian Bahreininejad 2835394

Tjeerd Rootmensen 2825571

Deniz Cinar Hykel 2814439

Zelal Derman 2836855

Öykü Bağdatoğlu 2844783

Date: 15.12.2024

What is the problem we are solving?

In this project we are given a layered problem with multiple aspects that need to be covered. To sum it up in one sentence the main idea is that we are creating a programme that assists the user with stock trading. To do this the programme will take several parameters called preference settings, rules and guidelines so to say, that are given to it by the user. In this case there are three preference settings that are taken into account. Firstly the “highest in the industry” parameter. This preference setting, when active, makes the programme only recommend stocks that are the highest performers in their respective industry. The second preference setting is called the “establishment year setting”. This setting is a guideline that makes sure the user only invests in well established, time proven companies. It does so by taking a year between 1800 and 2020 as an input from the user and only recommending companies that were established before the given year. Finally the third setting is called the “ESG criteria”. ESG stands for “environment, social, governance” these criteria make sure that the user only gets trading options recommended that use business practices that abide by these criteria. If this setting is active this is taken into account. One potential consideration we will have to make is what algorithm we will use to sort the stocks by ESG criteria as not all aspects of ESG should be given the same weight. Circling back to the main problem we see that all these factors combine into one big project that focuses on responsible investing, by taking the three criteria into account.

How we intend to split and organize the workload. We intend to split the workload by first writing a comprehensive outline of the tasks that are required for the project. Every group member will then select tasks according to their strengths or interests. For example, someone skilled in research could choose to write the context task. Similarly, someone interested in algorithms could focus on designing them. To make sure we finish the project on time, we intend to meet every 3-5 days to discuss the progress we have made and whether we need to address any challenges.

Context task

Modern computation and information technology (IT) has significantly impacted and transformed the financial world such as how individuals and companies are able to handle and transaction money and manage investments.

In the last few years there has been a big rise in algorithmic trading. Algorithmic Trading is a modern and recently developed computational system developed in the world of finance where systems carry out trades at a relatively high speed based on automated and programmed strategies. Since the implementation of Algorithmic Trading there has been an increase in the efficiency of the financial market. However, as well as that since the implementation of Algorithmic Trading, it has caused problems to the stability of the financial market which can be seen during the “Flash Crash” incident which was caused by algorithmic error in the system. [1] Cryptocurrency is another modern and significant computation system that is used and developed in the world of finance. Cryptocurrency introduces digital currencies such as Bitcoin, Binance and Ethereum to the financial systems and since the introduction of these digital currencies there has been less dependence on banking systems. However, Cryptocurrency presents challenges and problems of its own such as environmental concerns as Cryptocurrency has a very high energy mining process. [2]

Regardless of the challenges that computational and informational technologies impose, these innovations have both a positive and negative effect on the environment and their surroundings. Computational and informational technologies can provide access and be useful to financial

markets and individuals who work and manage their investments through trading platforms which can be seen as a positive impact on the social environment and consequences. However, the dependence on the complex computational and informational technologies in the finance world can have a negative impact on the social environment as the complexity of these technologies can create problems for individuals or groups that lack knowledge or access to these technologies. Furthermore the implementation of these technologies provides significant risks to data privacy.

How the problem was approached

We approached the problem methodically by using the *break down the problem into several subproblems* strategy. This gave us these problems to consider:

Determining the scope and constraints of our program. This included considering what kind of user interface we want to implement – graphical or text-based – and the extent of user customization. Ultimately, a text-based interface was chosen for its efficiency of development and simplicity, allowing users to toggle settings without feeling overwhelmed by the number of choices.

Choice of sorting algorithms. For sorting the database, our initial implementation utilized the bubble sort algorithm due to its simplicity. However, during testing, it became clear that bubble sort takes too long to finish sorting large datasets. This issue is further detailed in the testing section below.

Development of algorithms to filter the dataset. This required thinking about how the main parts of the project would function. We wanted to ensure that all filtering methods would work when combined, regardless of the order or combination in which they were selected. Additionally, the system needed to handle scenarios where the selected preference settings resulted in an insufficient number of stocks being generated. The final version of the algorithms was developed by first designing and refining them manually using pen and paper to ensure their logical consistency before writing the actual code.

Explanation of how our solution works

Our code is split into 3 main classes:

- **PreferenceSettings()** - Handles user input by storing the user's choices in attributes, allowing for easy access wherever it is needed in the code. Also handles text outputs, informing the user of how to select a setting and proceed.
- **Stocks()** - Represents individual stocks and stores their information in attributes. An instance of this class is created for every stock in the CSV file allowing for a structured and organized storage of stocks.
- **StockDatabase()** - This class contains functionality for storing and managing a database of stocks, which is implemented as just a list. It contains all the filtering methods and any additional methods that are required to process the data.

The filtering methods inside the StockDatabase class include

- `apply_establishment_year_filter()`
- `apply_industry_sort()`
- `assign_score_and_sort()`

These methods are designed to be as separate from one another as possible to help with debugging and easier understanding of the code. Detailed explanations are provided in the pseudocode and flowcharts in the appendix of this document.

Problems and difficulties encountered

We faced several problems during the development of the code.

In the initial version of the program, **insufficient planning led to bugs and inefficiencies** in our code. This resulted in significant debugging efforts because stock recommendations would not align with preference settings chosen by the user. We addressed this issue by tracing the flow of data manually using a pen and paper and adjusting the algorithm where necessary. This approach allowed us to gain insight into where potential bugs and inefficiencies could occur as we could consider edge cases more carefully and develop the algorithm beforehand which shortened development time overall. After that, a lot of the code relating to how the preference settings were applied had to be rewritten to produce a working version.

The order in which the preference settings were applied caused inaccuracies in stock recommendations, by either not being sorted properly in the final output or incorrect stocks being recommended. We fixed this by separating the methods that are responsible for applying the preference setting filters as much as possible, making them less dependent on one another. This involved rewriting the method `apply_industry_sort()` to also have an establishment year filter applied inside of it, called `filter_using_est_year()`. But there is also another method called `apply_establishment_year()` which gets called if the establishment year setting alone is chosen. This introduced redundancies which could potentially be considered a bad programming practice as it does not keep our code “DRY” (Don’t Repeat Yourself). However, it did fix our issue without requiring a complete rewrite of the filtering functions, thus providing a working and timely solution.

Handling cases when insufficient amounts of stocks are generated. When users apply the establishment year filter with an overly strict threshold, it may result in too few stock recommendations. For example, entering the year 1800 would result in only 2 stocks being recommended. If the user enters that they want 10 stocks, this would clearly fall short. How we addressed this issue was by using an iterative approach to calling the filtering functions with progressively higher threshold values until a sufficient amount of stocks could be recommended. While this approach is computationally expensive, it did not result in any noticeable performance decrease if merge sort was employed.

Determining ESG weights. We decided that environment, social, and governance (ESG) scores should not be considered equally. Hence, we applied different weights to each of these and then combined them using this formula:

$$score = performance + social \times s_{weight} + environment \times e_{weight} + governance \times g_{weight}$$

For each stock, a unique score is calculated based on its ESG values, which is then used to sort the stocks instead of relying only on the performance. One challenge we faced was how we would determine the individual weights. We considered hardcoding these values, however this approach would only work if the trader has prior programming knowledge and could therefore change the values themselves. As this is not a realistic assumption, we integrated the weight selection process as a part of the text interface. The user gets prompted to enter the weights if they enable the ESG preference setting. While this makes the text interface slightly more cluttered, the alternative – implementing a graphical user interface – would have significantly increased development time, making it an impractical choice for this project.

Testing the solution and changes made

We evaluated the efficiency of bubble sort to determine its ability to handle large datasets. To test this, we expanded the dataset to include several thousand stocks by duplicating existing entries multiple times. This resulted in repeated stock entries, however this was not a concern since our goal was to assess how well bubble sort could hold when used with a large dataset, compared to a faster algorithm such as merge sort.

Unsurprisingly, once the dataset reached around 5,000 stocks, bubble sort took a long time to sort the database whereas merge sort maintained consistently high performance. Considering there are an estimated 55,000 tradable stocks worldwide, we wanted to make sure that our implementation of iTrade could handle a database of such size. Sorting 55,000 data points with bubble sort took unreasonably long, but merge sort still performed well, managing to sort the database in under a second. The data gathered during this testing process is contained in Table 1.1. For this reason, we chose merge sort to sort the database of stocks.

Other decisions made

One decision that we considered was **whether to exclude negatively performing stocks**. From a beginner investor's perspective trying to avoid stocks that are guaranteed to lose money would seem logical. However we opted to keep them in to account for scenarios where the stock may recover or perhaps it would be within the interest of the investor's broader portfolio strategy to invest in them.

Displaying the output. We decided to always group the stocks by industry in the output. This approach was chosen to enhance the clarity of the recommended stock list, as it provides a logical distinction and visual separation between different stock categories.

Using a deque instead of a list. The decision was made to use a deque instead of a list when applying the industry sort in combination with an establishment year, as deques have a $O(1)$ time complexity for popping elements from the left side, compared to the $O(n)$ time complexity of regular Python lists. The left side of the dataset contains the best-performing stock based on the selected preference settings. This approach improves the speed of creating the final recommendations. Also worth noting is that deques aren't used in other parts of the code due to their larger memory requirements.

How did we split the tasks? Who did what? How much time did we spend on it?

Work division:

- **Arian:** Context task and project challenges (4 hours).
- **Deniz:** Designed the algorithm, wrote pseudocode, and tested the solution (10 hours).
- **Oyku:** Documented difficulties and solutions encountered (3 hours).
- **Zelal:** Coordinated group contributions, summarized task division, and managed timelines (3 hours).
- **Tjeerd Rootmensen:** Problem/project introduction and slide organization (3 hours)

Collaborative Efforts:

- **Flowchart design:** All members contributed (2 hours).
- **Testing and feedback:** Conducted by the group collectively (2-3 hours).

References:

[1] A. A. Kirilenko and A. W. Lo, 'Moore's Law versus Murphy's Law: Algorithmic Trading and Its Discontents', *Journal of Economic Perspectives*, vol. 27, no. 2, pp. 51–72, Feb. 2013, doi: 10.1257/jep.27.2.51.

[2] P. Howson and A. De Vries, 'Preying on the poor? Opportunities and challenges for tackling the social and environmental threats of cryptocurrencies for vulnerable and low-income communities', *Energy Research & Social Science*, vol. 84, p. 102394, Feb. 2022, doi: 10.1016/j.erss.2021.102394.

Appendix

Table 1

Data points	Bubble Sort time (s)	Merge Sort time (s)
2500	0.32	0.005
5000	1.61	0.012
7500	3.74	0.014
10000	6.53	0.017
12500	9.98	0.021
55000	220	0.087

Pseudocode 1:

```

1  1. Applying preference settings
2
3  IF the user selected the ESG preference setting THEN
4    EXECUTE FUNCTION assign_score_and_sort
5  ENDIF
6
7  IF the user selected the highest in industry preference setting THEN
8    EXECUTE FUNCTION apply_industry_sort
9  ELSE
10   IF the user selected establishment year preference setting THEN
11     EXECUTE FUNCTION apply_establishment_year_filter
12   ENDIF
13
14  IF the user didn't turn on any preference settings:
15    sort the dataset based on each stock's performance
16    STORE only the top_n stocks from the database and ignore the rest
17  ENDIF

```

Pseudocode 2:

```
1  2. Assign score and sort function
2
3  FUNCTION assign_score_and_sort
4      FOR every stock in database D0
5          SET score TO stock's performance + stock's E rating * \
6              e_weight + stock's S rating * s_weight + stock's G rating + \
7              g_weight
8          Assign the score to each stock using the above calculation
9
10     # Sort the dataset by the score attribute
11     Sort the dataset based on the stocks' score value
12 END FUNCTION
```

Pseudocode 3:

```

1  3. Highest in industry preference setting
2
3  FUNCTION apply_industry_sort(top_n, est_thresh)
4      # Initialize the variable to contain the stocks in the dataset grouped by their industry
5      INITIALIZE VARIABLE industries_dataset = restructure_ds_by_industry()
6
7      FUNCTION filter_using_est_year(dataset, threshold)
8          FOR EACH industry IN dataset DO
9              FILTER stocks in industry IF their foundation_year <= threshold
10             END FOR
11             RETURN dataset
12         END FUNCTION
13
14         FUNCTION sort_remaining_stocks(dataset)
15             FOR EACH industry IN dataset DO
16                 SORT stocks belonging to this industry by their score
17             END FOR
18             RETURN dataset
19         END FUNCTION
20
21         # Apply a filter to each of the industries, removing stocks that fall below it
22         IF est_thresh has a value, meaning the user turned on that preference setting
23             EXECUTE FUNCTION filter_using_est_year(industries_dataset, est_thresh)
24         ENDIF
25
26         # For each of the industries, sort the stocks that passed the filter
27         EXECUTE FUNCTION sort_remaining_stocks(industries_dataset)
28
29         # Initialize a list for picked stocks and set iteration counter
30         INITIALIZE VARIABLE picked_stocks = empty list
31         INITIALIZE VARIABLE iterations = 1
32         INITIALIZE VARIABLE threshold_changed = False
33
34
35         # Continue until top_n stocks are selected
36         WHILE the length of picked stocks is less than top_n DO
37             FOR EACH industry, stocks IN industries_dataset DO
38                 TRY
39                     # Pop the top stock and add to the picked stocks list
40                     SET top_stock = POP_LEFT from stocks
41                     ADD top_stock TO picked_stocks
42                 IF an IndexError occurs:
43                     # Not enough stocks, have to increase threshold by 1 year and try again.
44                     SET industries_dataset = restructure_ds_by_industry()
45                     SET industries_dataset = filter_using_est_year(industries_dataset, est_thresh + iterations)
46                     SET industries_dataset = sort_remaining_stocks(industries_dataset)
47                     REMOVE all elements from picked_stocks
48                     SET threshold_changed = True
49                 END FOR LOOP HERE
50             ENDIF
51
52             # Increment for next iteration
53             SET iterations += 1
54         END WHILE
55
56         IF the threshold had to be changed
57             OUTPUT warning message to user
58         END IF
59
60         # Set the final dataset to the stocks that were picked
61         SET dataset = picked_stocks
62     END FUNCTION

```

Pseudocode 4:

```

1   4. Apply establishment year filter
2
3   ∨ FUNCTION apply_establishment_year_filter(top_n, threshold_year)
4       # Initialize the variables
5       SET increase_threshold TO False
6       SET iterations TO 0
7
8       # Keep looping until enough stocks are selected
9   ∨ WHILE increase_threshold is False DO
10      # Retrieve the original dataset
11      SET self.dataset TO GET_ORIGINAL_DATABASE()
12
13      # Sort the dataset by score
14      EXECUTE FUNCTION SORTING_ALGORITHM(self.dataset, "score")
15
16      # Filter the dataset based on the threshold year and iterations
17      INITIALIZE VARIABLE filtered_dataset = empty list
18  ∨   FOR EACH stock IN dataset DO
19  ∨       IF stock.foundation_year <= threshold_year + iterations THEN
20  ∨           ADD stock TO filtered_dataset
21  ∨       END FOR
22
23      # Check if enough stocks have been selected
24  ∨   IF the length of filtered_dataset is greater than or equal to top_n THEN
25  ∨       SET increase_threshold TO True
26  ∨   ENDIF
27
28      # Stop condition to avoid infinite loop
29  ∨   IF iterations == 300 THEN
30  ∨       STOP FUNCTION here
31  ∨   ENDIF
32
33      # Increase the threshold year for the next iteration
34      SET iterations TO iterations + 1
35  END WHILE
36
37      # Print warning message if the threshold year had to be increased
38  ∨   IF increase_threshold is True THEN
39  ∨       PRINT warning message: "Not enough stocks, increasing year
40  ∨       threshold to {threshold_year + iterations - 1}."
41  ∨   ENDIF
42
43      # Set the final dataset to the top_n stocks
44      SET self.dataset TO filtered_dataset[:top_n]
45  END FUNCTION

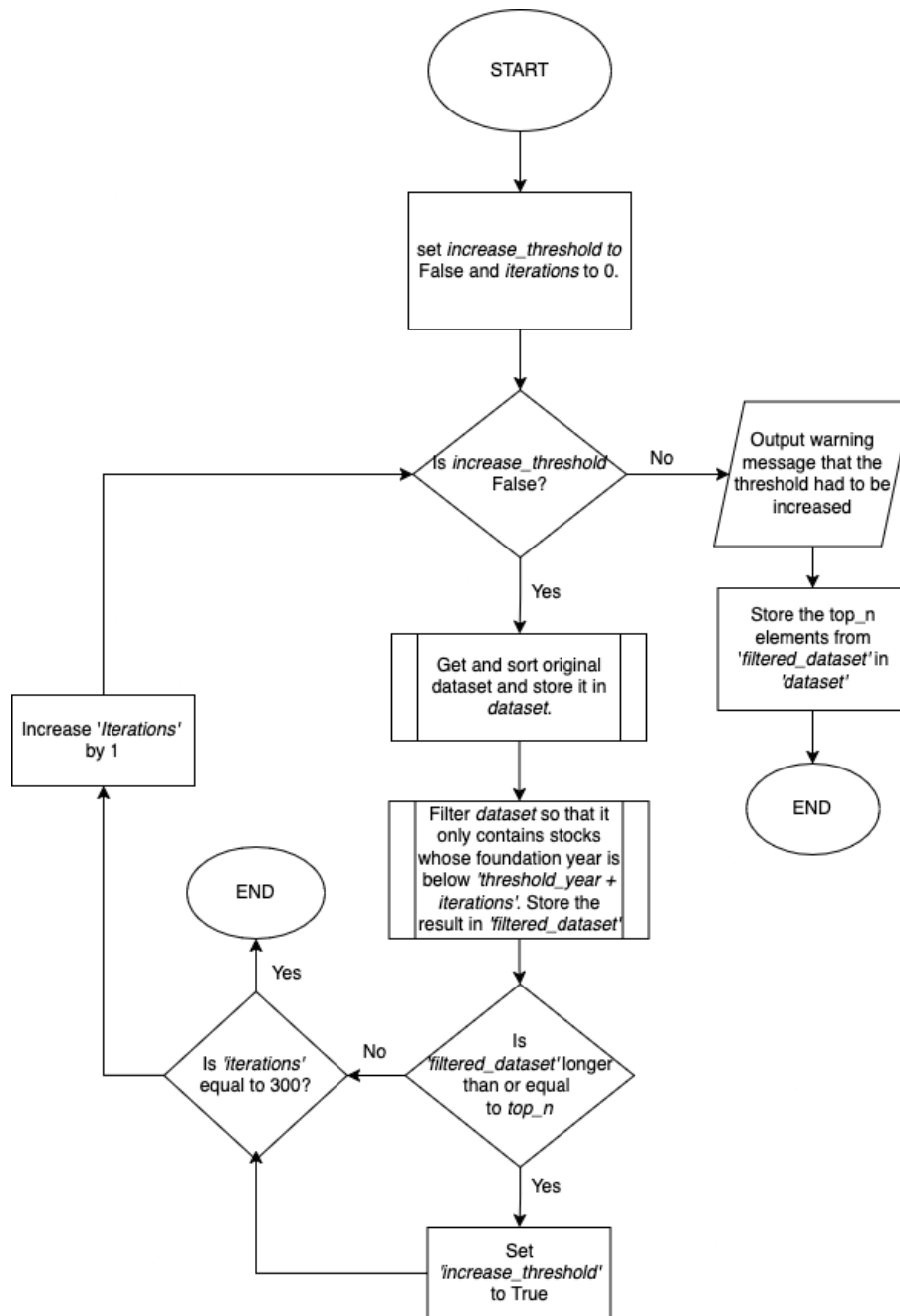
```

Pseudocode 5:

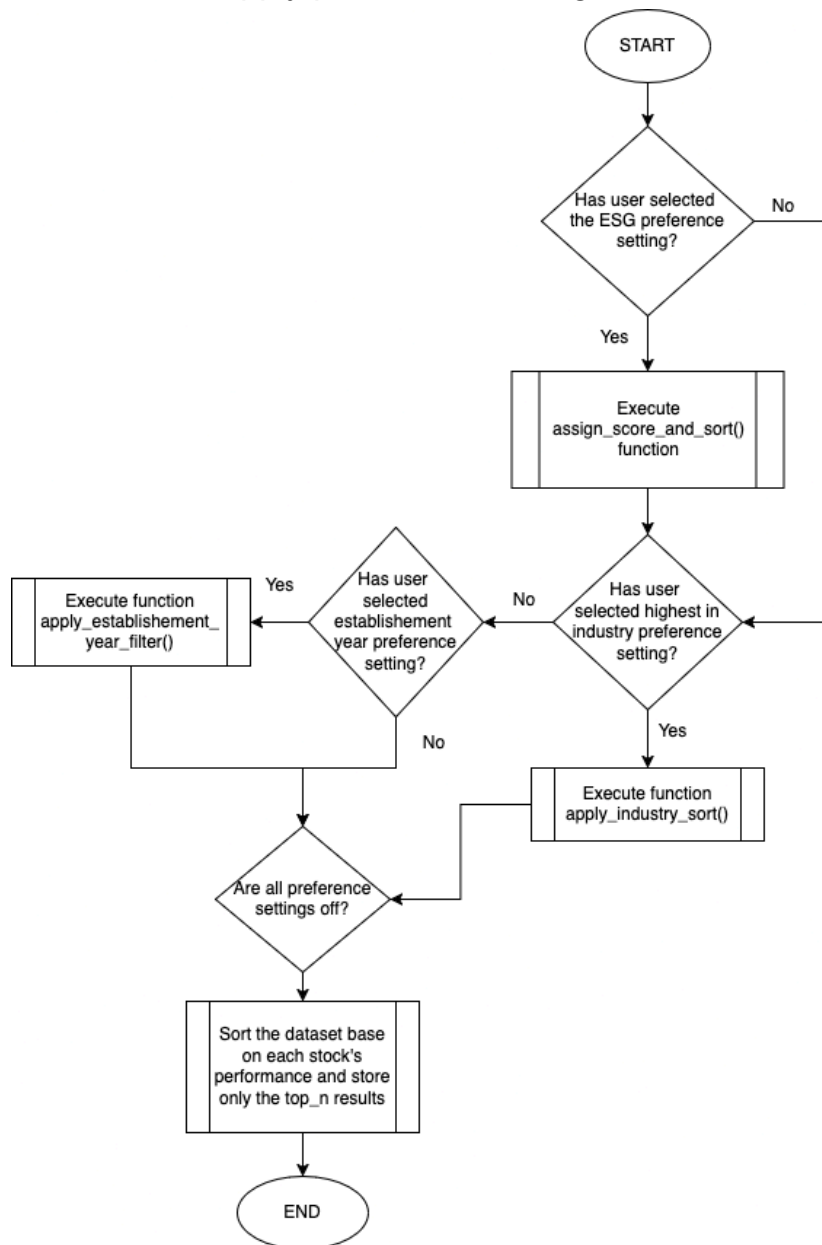
```
1  5. Validation of user input
2
3  WHILE TRUE:
4      TRY
5          PROMPT user with the message "how many stocks
6          |      to recommend" and store it in top_n
7          Attempt to convert top_n to an integer
8          IF top_n is less than 0 or more than 100 THEN
9              Print "Please enter a positive integer."
10             CONTINUE onto the next iteration of the loop
11         ENDIF
12     END LOOP HERE
13     CATCH error
14         Print "The value has to be an integer!"
15     END TRY
16 END WHILE
```

Flowcharts

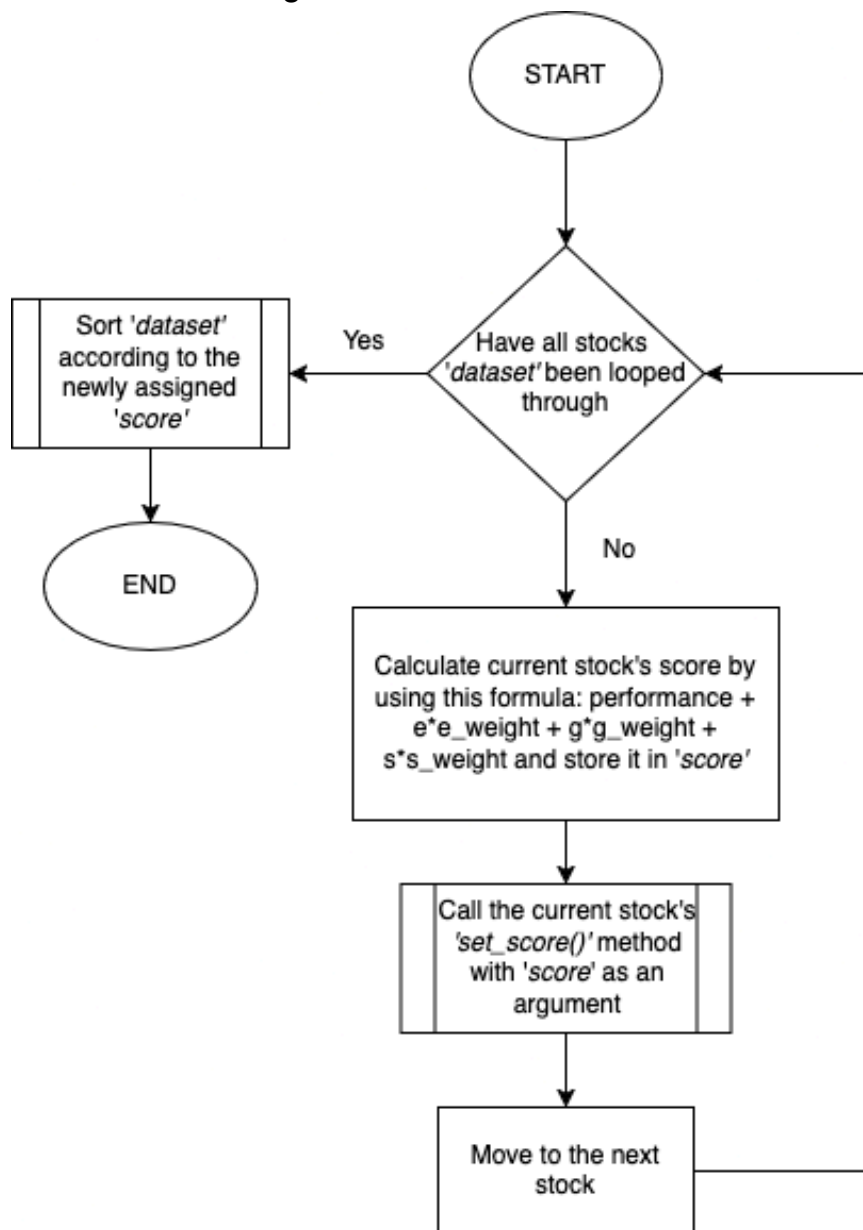
Flowchart 1 - Apply establishment year method flowchart:



Flowchart 2 - Apply preference settings flowchart:



Flowchart 3 - Assign score and sort method flowchart:



Flowchart 4 - Validation of user input flowchart:

